ENGR 102 Summer Bridge
Day 10 Student Workbook

Lectures 1-5 Recap, Direct Input, and Repetition Launch

Friday, July 17, 2026 • 50 points

Name		**UIN**		**Section**	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Daily target — increasing independence

Connect input, state, Boolean decisions, and testing to a first complete while-loop program.

Allowed tools	input, int, float, arithmetic, comparisons, Boolean operators `and`, `or`, and `not`, if/elif/else, assignment, print, and a first while loop after the quiz
Support supplied	Variable names, a floor-validity rule, the movement goal, and named boundary tests are supplied.
You must decide	The conversion, validation branch order, continuation condition, direction decision, and one-floor update.
Scaffold level	Planning frame supplied; the complete elevator loop is student-authored.

Scoring and completion standard — 50 points

Evidence	**Points**	**Secure work shows**
Integrated trace	14	intermediate state, condition results, and exact output
Diagnosis and repair	12	actual, intended, cause, repair, and a boundary test when requested
Full program and tests	24	coherent executable logic, required output, and defended tests

Independence standard. You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

Task 1. Integrated trace — 14 points

Trace types, Boolean subexpressions, and branch order. Do not jump directly to the final line.

```
sample = "14"
offset = 5
value = int(sample) * 2 + offset
stable = 20 <= value <= 35 and value % 3 == 0

if not stable:
    status = "review"
elif value >= 30:
    status = "high"
else:
    status = "normal"

print(value, stable, status)
```

Your task

1. Compute value and identify its type. Then evaluate each side of the **and** expression used to compute stable.
2. State which branch assigns status and explain why the earlier branch does not execute.
3. Write the exact printed output, including the Boolean capitalization.

A final answer without the requested state evidence cannot earn full trace credit.

Task 2. Diagnose and repair **12 points**

The requirement is: approval requires training, and it also requires either a temperature from 15 through 30 inclusive or a supervisor override.

```
trained = False
temperature = 20
supervisor = False

approved = trained and temperature >= 15 or temperature <= 30 or supervisor
print(approved)
```

Your task

1. Show the actual Boolean result for the supplied values and explain why it violates the requirement.
2. Write one corrected Boolean assignment and defend its grouping with a test in which trained is False.

Debugging evidence is complete only when the repair is tied to the stated defect and an expected result.

Task 3. Construct a complete program — 24 points across Pages 4-5

Program specification

- Read current_floor and desired_floor as integers.
- Valid floors are 1 through 10 inclusive. Print invalid if either input is outside that interval.
- If the valid floors are equal, print already there.
- Otherwise, use a while loop that changes current_floor by exactly one floor per pass until it equals desired_floor.
- Print each newly reached floor inside the loop. Do not jump directly to the destination.

Required planning evidence

1. Write the invalid-input Boolean expression and the branch order that protects the loop.
2. Write the continuation condition and both possible one-floor updates before coding.

Program workspace – begin the connected solution here.

Task 3 continued. Test and defend the program — included in 24 points

Continue code if needed.

Required tests, expected results, and brief defense

Copyright © 2026 Lance L.A. White. All rights reserved.